

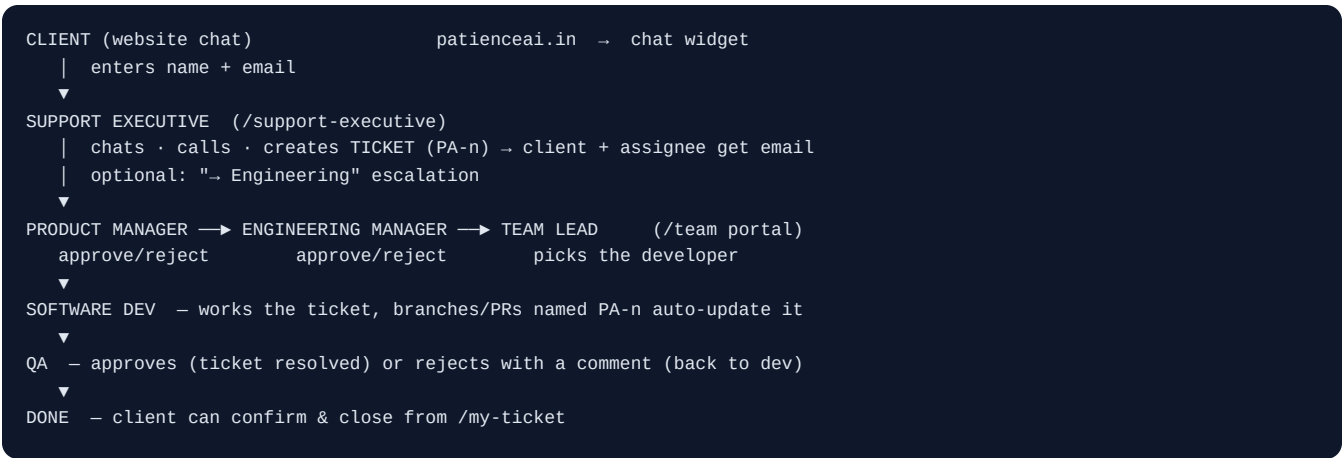
PatienceAI — Team Guide & Working Workflow

Share this document with everyone. It explains every portal, role, workflow and endpoint in the platform — from a client opening a chat to a developer merging a pull request.

Last updated: June 2026 · Stack: React + Vite · Express · Neon Postgres · Redis · Cloudflare R2 · GitHub API · Groq AI

1. The Big Picture

A ticket is the center of everything. Support conversations become tickets; tickets flow through product/engineering; GitHub work, QA, sprints, incidents and emails all attach back to the ticket.



Missing tiers auto-skip (no EM yet? PM approval goes straight to the Team Lead).

2. Portals & Who Uses Them

URL	Who	What they do
/	Public	Company website + AI chat widget ("Talk to a live agent" starts support)
/support-executive	Support executives	Live chats, voice calls, transfers, internal team chat, create tickets , tickets workspace, escalate to engineering
/team	Software team (dev, lead, EM, PM, QA) + generic members	My tickets bucket · Engineering workspace (pipeline, sprints, epics, incidents, QA cases, OKRs, services, announcements) · GitHub workspace (permission-gated) · dark/light theme
/my-ticket	Clients (no login)	Track status with ticket key + email, reply, upload files, confirm-close
/admin	Admin	Everything: analytics, site content, blog, submissions, AI conversations, live support, executives , team (invite + roles + permissions), tickets (SLA/categories/KB/audit/performance), engineering (full PEOS + GitHub console)

Logins: executives and team members are invite-only (email link → set password). Clients never log in — they prove ownership with `PA-n` + their email.

3. Roles & Permissions (set by Admin)

Assigned in **admin** → **team** when inviting (or changed later inline):

Team role	Ticket workflow powers	Engineering workspace	GitHub default
software_dev	Complete dev work → send to QA	View all; create incidents & QA cases	read + write (branches, PRs)
team_lead	Assign tickets to developers	Full edit of sprints/epics/OKRs/services/announcements + delete	read + write
engineering_manager	Approve EM stage; assign	Full edit + delete; roster manage	read + write
product_manager	Approve/reject PM stage; escalate	Full edit + delete; roster manage	read
qa	QA approve / reject-with-comment	View; create/run QA cases & incidents	read
member	Work assigned support tickets only	View only	none

Per-user overrides: admin clicks the permission chips (`github read` / `github write` / `roster manage`) on any member row — overrides the role default for that person. An explicitly emptied set means *no* permissions.

Admin and Support Executives bypass all gates. PM/EM can also invite & manage roster (not delete accounts).

4. Day-to-Day Workflows

4.1 Support ticket (the foundation)

1. Client chats on the website → executive answers in `/support-executive` .
2. Executive clicks **Create ticket** inside the chat (pre-filled with the client) or **New ticket** in the Tickets workspace. Picks category, priority, assignee (`@patienceai.in` only — most-used suggested), attaches files.
3. Automatic: client gets a confirmation email with a `/my-ticket` link; assignee gets a "task assigned" email + in-app notification; delivery status recorded on the ticket.
4. **SLA clock starts:** urgent 4h / high 12h / medium 24h / low 72h (admin-editable). Countdown badges everywhere; warnings before the deadline; breach alerts after.
5. **Escalation ladder if assignee is silent:** reminder → raising executive → admin (every step recorded + emailed).

4.2 Engineering pipeline (Jira-style)

1. Executive (or PM) presses → **Engineering** on a ticket → lands in the **PM review** bucket.
2. PM approves (→ EM / Lead) or rejects with a mandatory comment (→ back to support).
3. Team Lead types the developer's email → ticket lands in the **dev's bucket** (status: in progress).
4. Dev works it — comments, internal notes, attachments — clicks **Complete** → **QA**.
5. QA approves (→ done, resolved) or **Send back** with a required improvement comment (→ dev again).
6. Action buttons appear automatically on the ticket in `/team` based on **your role + the ticket's stage**. The whole pipeline is visible as a board in the **engineering** view.

4.3 GitHub (developers)

- Name branches/PR titles with the ticket key: `PA-12-fix-login` .
- **Automatic** (via webhook): branch/commit/PR mentioning `PA-n` links to the ticket, posts on its timeline, notifies the assignee, and advances the workflow (branch/PR-open → in progress, **PR merged** → **resolved**).
- **Manual** (in `/team` → github, or admin → engineering → github): browse repos, create/delete branches, **create / merge / close pull requests** — write actions need the `github_write` permission.

4.4 Client self-service

- Email link → `/my-ticket` → live status + timeline (staff names & internal notes hidden), reply, upload files (≤10 MB any format), **"My issue is resolved — close"** with confirmation.

4.5 QA, incidents, sprints, OKRs (engineering workspace in /team and admin)

- **QA test cases:** link to PA-n , record Pass/Fail/Blocked runs.
- **Incidents:** SEV-1..4, investigate → resolve → postmortem → close.
- **Sprints:** plan capacity, start/finish; ticket story points feed the velocity KPI.
- **Service catalog:** owners, runbooks, SLAs, dependency map (uncataloged deps flagged red).
- **OKRs:** company → department → team → sprint with progress bars.

5. Complete API Reference (also machine-readable at /api/openapi.json)

Auth = session cookie unless noted. PA-n accepted anywhere a ticket id is.

Tickets — api/tickets.js

Endpoint	What
GET /api/tickets	List. Filters: status, priority, category, assignee, clientEmail, dateFrom, dateTo, ticketId, search · ?id= single (with comments/attachments/escalations) · ?suggest=1 assignee suggestions · ?export=csv
POST /api/tickets	Create (exec/admin) — sends client + assignee emails, sets SLA due_at
PATCH /api/tickets	Update status/priority/assignee — {id} or bulk {ids:[]}
DELETE /api/tickets	Delete (staff)
POST /api/tickets/comments	Comment; {isInternal:true} = staff-only note; @name mentions notify

Attachments (Cloudflare R2) — api/attachments.js

POST /api/attachments/upload?ticketId&fileName[&clientEmail]	Raw file body, ≤10 MB, any format
GET /api/attachments?id=	Download → 302 to presigned R2 URL
GET /api/attachments?ticketId=	List metadata

Client portal (public, rate-limited) — api/client-tickets.js

GET /api/client-tickets?key=PA-n&email=	Status + public timeline
POST /api/client-tickets	{key,email,message} reply · {key,email,action:"close"} confirm-close

Team accounts — api/team-members.js

POST /login · activate · change-password · GET /me · DELETE /logout	Member auth (password strength enforced)
GET/POST/PATCH/DELETE /api/team-members	Roster (admin + PM/EM; delete admin-only). POST takes teamRole ; PATCH takes status , teamRole , permissions:[]

Dev workflow — api/dev-workflow.js

GET /api/dev-workflow	Pipeline grouped by stage (?bucket=1 = only mine) + myRole
POST /api/dev-workflow	{ticketId, action, comment?, assigneeEmail?} — actions: escalate · pm_approve · pm_reject · em_approve · em_reject · lead_assign · dev_complete · qa_approve · qa_reject (role-gated; rejects require comment)

PEOS resources — api/peos.js

GET/POST/PATCH/DELETE /api/peos/ resource=	epics · sprints · incidents · services · okrs · announcements · testcases (write: PM/EM/Lead manage everything; dev/QA create incidents & test cases; delete: managers/leads)
GET ?dashboard=1	Health score, open/overdue/breaches, critical incidents, velocity
GET ?search=	Universal search: tickets, incidents, services, epics, docs, people, GitHub links
GET ?sprintBoard=ID · ? githubFor=ID · ?summarize=PA-n (AI)	Sprint board · GitHub links · AI ticket summary
PATCH ?ticket=PA-n	Plan: {sprintId, epicId, storyPoints} (managers/leads)

GitHub — api/github.js + api/github-webhook.js

GET /api/github	?status=1 connection · ?repos=1 · ?branches=1&owner&repo · ?prs=1 · ?commits=1
POST /api/github? owner&repo	create_branch {branch,from} · create_pr {title,head,base} · merge_pr {number} · close_pr {number} · delete_branch {branch} · request_review — needs github_write
POST /api/github- webhook	GitHub → us (push/PR/release). HMAC-verified. PA-n auto-linking + workflow progression

Supporting

GET/PATCH /api/notifications	Feed + unread count · mark read
GET /api/ticket-settings (+admin writes)	SLA hours, categories, saved responses
GET /api/ticket-stats	Performance dashboard (?export=csv) · ?audit=1 admin audit log
GET /api/kb?search= (+admin CRUD)	Knowledge base; suggested while typing a ticket subject
GET /api/openapi.json	This table, as OpenAPI 3

Plus pre-existing: /api/support-chat , /api/support-executives/* , /api/voice-room , /api/auth , /api/chat , /api/contact , /api/analytics , /api/site-content , /api/newsletter .

6. File Map (where everything lives)

api/	
tickets.js	ticket CRUD, filters, bulk, CSV, emails, SLA stamp
tickets → comments	(same file) comments, internal notes, mentions
dev-workflow.js	PM-EM-Lead-Dev-QA pipeline, role gates
team-members.js	member auth, roster, roles, per-user permissions
peos.js	epics/sprints/incidents/services/okrs/announcements/ testcases + dashboard + universal search + AI summary
github.js	outbound GitHub (repos/branches/PRs, write actions)
github-webhook.js	inbound auto-linking + workflow progression
attachments.js	R2 uploads/downloads (10 MB, any format)
client-tickets.js	public client portal API
notifications.js	in-app notification feed
ticket-settings.js	SLA rules, categories, saved responses
ticket-stats.js	performance dashboard + audit log
kb.js	knowledge base
_escalation.js	5-min sweep: reminders, escalation ladder, SLA breach
_ticketing.js	audit log, notify(), mentions, seeds, SLA lookup
_cache.js	Redis read-cache + version-stamped invalidation
_redis.js	persistent-connection RESP client (dependency-free)
_r2.js	Cloudflare R2 helper (put / presigned get)
_ai.js	AI provider abstraction (Grok default)
_openapi.js	the OpenAPI contract
_db.js	Neon client + full schema migrations + query counter
_security.js	sessions (admin/exec/member), script passwords
src/pages/	

```

SupportExecutivePage.jsx    exec console (chat, calls, tickets, escalate)
TeamPortalPage.jsx         /team: my tickets · engineering · github views
ClientTicketPage.jsx       /my-ticket
AdminPage.jsx              admin tabs incl. team (roles+perms) and tickets

src/components/
TicketCenter.jsx           ticket UI: modal, detail, filters, bulk, bell, SLA
TeamEngineering.jsx        engineering workspace inside /team
AdminTicketOps.jsx         admin: performance, SLA editor, KB, audit
AdminPeos.jsx              admin: engineering tab (dashboard, GitHub console)

server.js                  routes, security headers, escalation timer
documentation/12_ticketing_system.md  deep architecture doc
documentation/TEAM_GUIDE.md    this file

```

7. Performance & Reliability (how it stays fast and cheap)

- **Redis read-cache:** every polling read (lists, details, notifications, settings) is served from Redis with version-stamped keys — a write bumps a counter so the next poll is instantly fresh. Measured: **36 polled requests** → **0–6 Postgres queries** (~95–100 % reduction). Protects the Neon free tier and bandwidth.
- **R2 file storage:** file bytes never touch Postgres or the app server on download (presigned URLs).
- **Graceful degradation:** Redis down → direct DB reads. R2 unset → small files in DB. GitHub unset → clear "connect" hint. Email down → ticket still created, failure recorded.
- Sessions: HMAC-signed cookies, 7-day expiry; scrypt password hashing; login rate-limiting; webhook HMAC verification; audit log on every sensitive action.

8. Honest Implementation Ratings (no bluff)

Area	Rating	Honest notes
Ticketing core (SLA, escalation, emails, attachments)	9/10	Battle-tested end-to-end incl. failure paths. Missing: per-ticket SLA pause/holiday calendars.
Dev workflow pipeline	8.5/10	Full ladder tested with real role accounts incl. QA reject loop. Routing picks the <i>*first*</i> active person per role — no round-robin/load balancing yet; one pipeline (no per-team pipelines).
Roles & permissions	8.5/10	Role defaults + per-user overrides + revocation edge case fixed and unit-tested. Permissions are 3 flags, not a full policy engine.
GitHub integration	8/10	Webhook auto-linking + full console verified against the real repo (branch create/delete live-tested). Single shared PAT — actions aren't attributed to individual GitHub accounts (they are in our audit log). PR review threads not surfaced.
Redis caching	9/10	Rewrote a genuinely broken client (the old one corrupted every JSON read); measured near-zero DB load. Remote-Redis latency can let short TTLs lapse (a few stray queries — harmless).
R2 attachments	9/10	10 MB/any format verified byte-identical; 413 + type-fallback paths tested. No virus scanning.
Client portal	8.5/10	Privacy verified (internal notes/staff names never leak; wrong email → 404). Auth is key+email — fine for support, not bank-grade.
PEOS (sprints/epics/incidents/OKRs/services/QA)	7.5/10	Solid CRUD + dashboards + dependency map, all role-gated. Burndown <i>*charts*</i> , drag-drop boards, and a rules-builder UI don't exist yet — velocity/health are simple aggregates.
AI copilot	7/10	Provider-abstracted, working summaries. No auto-assignment/duplicate detection yet (schema is ready).

Tests	7/10	Extensive scripted API/integration tests run at every step (all documented in commits), but they're not committed as a CI suite — no automated regression net.
Overall	8.5/10	Production-ready internal platform; the honest gaps are listed above and all additive.

9. Setup Checklist (Render env)

`DATABASE_URL` · `REDIS_URL` · `ADMIN_USERNAME/PASSWORD/SESSION_SECRET` · `BREVO_API_KEY` + `BREVO_SENDER_EMAIL` (or `SMTP_*`) · `R2_ACCOUNT_ID/ACCESS_KEY_ID/SECRET_ACCESS_KEY/BUCKET_NAME` · `GITHUB_TOKEN` + `GITHUB_OWNER` + `GITHUB_WEBHOOK_SECRET` · `GROQ_API_KEY` · `SITE_URL`

Optional: `TICKET_REMINDER_HOURS` · `TICKET_SLA_WARNING_HOURS` · `AI_PROVIDER` · `DB_QUERY_LOG`

GitHub repo webhook → `https://patienceai.in/api/github-webhook` (JSON, same secret, events: push + pull_request + release).